ISPARTA UYGULAMALI BİLİMLER ÜNİVERSİTESİ

ELEKTRİK ELEKTRONİK MÜHENDİSLİĞİ

MAKİNE ÖĞRENMESİ SÜREÇLERİ VE MODEL DEĞERLENDİRME STRATEJİLERİ

Makine Öğrenmesine Giriş

HAZIRLAYAN

Hamdi Furkan YILMAZ - 2322705007

Ders Sorumlusu: Ali ŞENTÜRK

1. GİRİŞ

Klasik yazılım projelerinde, problemi çözecek kuralları oturup satır satır bizim yazmamız gerekir. Makine öğrenmesinde ise işin rengi değişiyor; algoritmaya veriyi veriyoruz ve kuralları kendi kendine bulmasını bekliyoruz. Günümüzde karşılaştığımız veri setleri o kadar büyük ve karmaşık ki, oturup her senaryoyu kapsayacak bir kural dizisi yazmak imkansız hale geldi. Biz de tam bu noktada makine öğrenmesini kullanarak verilerin içindeki o gizli bağları matematiksel olarak çözmeye çalışıyoruz. Domingos'un [1] da vurguladığı gibi, makine öğrenmesi sadece havalı algoritmalardan ibaret değil. Asıl mesele, elimizdeki ham veriyi sahada işe yarayacak bir sisteme dönüştürebilmek. Projelerde başımızı en çok ağrıtan şey de genelde bu oluyor: Geliştirme aşamasında harika çalışan, hata oranı sıfıra yakın olan bir model, daha önce hiç görmediği gerçek dünya verisiyle karşılaşınca çuvallayabiliyor.
Biz de bu raporda, bir makine öğrenmesi modelinin uçtan uca nasıl kurgulanması gerektiğini ele aldık. Özellikle modelin veriyi "ezberlemesinin" önüne geçecek veri bölme taktiklerine odaklandık. Ayrıca, sistemin ne kadar iyi çalıştığını ölçmek için sadece tek bir "doğruluk (accuracy)" yüzdesine bakmanın neden yetersiz kalacağını farklı metriklerle göstermeyi hedefledik.

2. TEMEL KAVRAMLAR

Proje boyunca model geliştirirken ve sonuçları analiz ederken şu temel kavramları merkeze aldık:

- **Makine Öğrenmesi İş Akışı (ML Pipeline)** Bir model tasarlamak sadece kod yazmaktan ibaret değil. Sculley ve ekibinin [2] "gizli teknik borç" makalesinde bahsettiği gibi, yazılan asıl makine öğrenmesi kodu tüm sistemin ufacık bir parçası. İşin asıl büyük yükü; veri toplama, veriyi temizleme, özellik çıkarımı (feature extraction) ve sonrasında sistemi izleme adımlarında. Veriyi iyi temizlemezsek veya veri setinde eksikler kalırsa, modelin performansı doğrudan bozuluyor.

- **Yanlılık ve Varyans Dengesi (Aşırı Öğrenme/Overfitting)** Modelin eğitim verisini öğrenmesiyle, yeni verilerde düzgün çalışması arasındaki dengeyi bulmak en kritik aşamalardan biri. Model çok basit kalırsa verideki mantığı hiç anlayamıyor (Underfitting / Eksik Öğrenme). Ama modeli çok karmaşıklaştırırsak, bu sefer de verideki gürültüleri bile ezberlemeye başlıyor (Overfitting / Aşırı Öğrenme) [1]. Eğitim setinde %100 başarı görsek de test setine geçince modelin başarısı ciddi şekilde düşüyor. Buradaki temel mühendislik problemi, modelin ne çok tembel ne de tam bir ezberci olduğu o ideal denge noktasını yakalamak.

- **Veri Bölme Stratejileri** Modelin gerçekten bir şeyler öğrenip öğrenmediğini anlamanın tek bir yolu var: Karşısına daha önce hiç görmediği veriler çıkarmak. Kohavi'nin [3] çalışmalarında da anlattığı gibi, elimizdeki veriyi Eğitim (Train) ve Test olarak ikiye ayırmak işin en temel kısmı. Uygulama tarafında genelde araya bir de Doğrulama (Validation/Dev) seti sıkıştırıyoruz. Model Train ile öğreniyor, Validation ile biz parametre ayarlarını (ince ayarları) yapıyoruz, Test setiyle de modelin nihai performansını ölçüyoruz.

- **Gelişmiş Veri Bölme: K-Fold Çapraz Doğrulama (Cross-Validation)** Statik bir eğitim-test ayrımı bazen verinin sadece belirli bir kısmına odaklanmamıza neden olabilir. Özellikle veri setinin kısıtlı olduğu durumlarda "hayat kurtarıcı" olan K-Fold yöntemiyle, veriyi K adet eşit parçaya bölüyoruz. Her iterasyonda bir parça "test", geri kalanlar ise "eğitim" seti olarak kullanılıyor. Kohavi'nin [3] belirttiği gibi, bu döngü sonunda elde edilen performans sonuçlarının ortalamasını almak, modelin gerçek dünyadaki başarısı hakkında bize çok daha kararlı ve güvenilir bir projeksiyon sunuyor.

- **Model Değerlendirme Metrikleri** Özellikle sınıfların dengesiz dağıldığı veri setlerinde "modelin doğruluk oranı çok yüksek" deyip geçmek yanıltıcı olabiliyor. Sistemin nerede hata yaptığını tam olarak görmek için Karmaşıklık Matrisi (Confusion Matrix) yapısını kullanıyoruz. Bu matris bize Gerçek Pozitif (TP), Gerçek Negatif (TN), Yanlış Pozitif (FP) ve Yanlış Negatif (FN) olmak üzere dört farklı bileşen

sunuyor. Hossin ve Sulaiman'ın [4] çalışmasında da belirtildiği gibi, modelin performansını daha şeffaf ve objektif ölçmek için aşağıdaki formülleri kullandık:

Doğruluk (Accuracy): Toplam tahminler içinde doğru yapılanların oranıdır.

$$Doğruluk\ (Accuracy) = \frac{TP + TN}{\mathrm{TP + TN + FP + FN}}$$

Kesinlik (Precision): Pozitif olarak tahmin edilen örneklerin ne kadarının gerçekten pozitif olduğunu gösterir.

$$Kesinlik(Precision) = \frac{TP}{\mathrm{TP + FP}}$$

Duyarlılık (Recall): Gerçekten pozitif olan durumların model tarafından yüzde kaç oranında başarıyla yakalandığını ölçer.

$$Duyarlılık(Recall) = \frac{\mathrm{TP}}{\mathrm{TP + FN}}$$

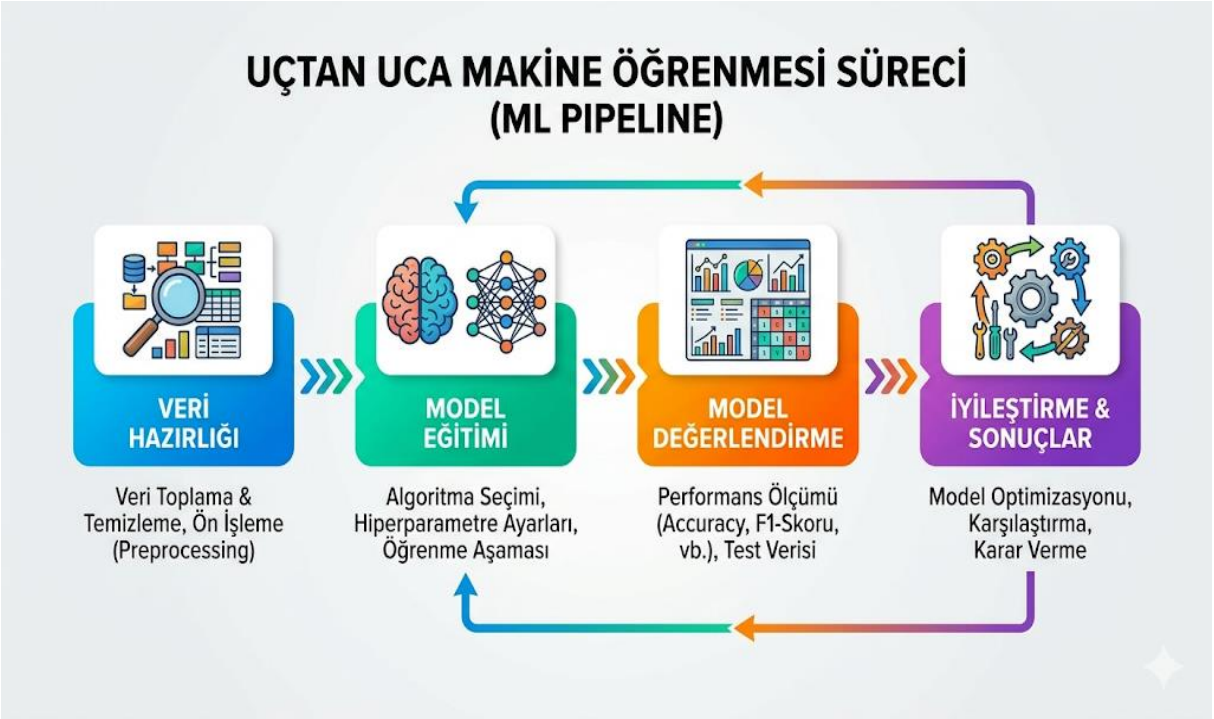
F1-Skoru: Kesinlik ve Duyarlılık değerlerinin harmonik ortalamasını alarak daha dengeli bir sonuç sunar.

$$F1\ Skoru\ = 2\ \times \frac{Precision \times Recall}{\mathrm{Precision + Recall}}$$

Regresyon Modelleri İçin Değerlendirme Metrikleri Sınıflandırma problemlerinde doğruluk oranlarına bakarken, sayısal tahmin (regresyon) gerektiren projelerde hedefin "%100 bilmek" değil, "sapmayı minimize etmek" olduğunu unutmamalıyız. Bu noktada iki temel metrik öne çıkıyor:

- **MAE (Mean Absolute Error - Ortalama Mutlak Hata):** Tahminlerimizin gerçek değerlerden ortalama ne kadar saptığını gösterir. Örneğin bir ev fiyatı tahmininde "ortalama 20 bin TL yanılıyoruz" dememizi sağlar.
- **MSE (Mean Squared Error - Karesel Orata Hata):** Hataların karesini aldığı için büyük hataları daha sert cezalandıran bir sistemdir. Modelin uçuk kaçık hatalar yapmasını engellemek ve hassasiyeti artırmak adına kritik bir ölçüttür.

3. YÖNTEM / MODEL AÇIKLAMASI



Sınıflandırma ve regresyon problemlerinde işimizi oldukça kolaylaştıran Random Forest algoritması, aslında tek bir devasa model kurmak yerine birçok karar ağacını bir araya getiren bir "topluluk öğrenmesi" (ensemble learning) yöntemi. Algoritmanın çalışma mantığı temel olarak şu adımlarla ilerliyor:

- **Bootstrap Örneklemesi (Bagging):** Elimizdeki ana eğitim veri setinden rastgele çekilişler yapıyoruz (seçtiğimizi yerine koyarak). Böylece birbirine benzeyen ama ufak farkları olan alt veri setleri üretiyoruz.
- **Karar Ağaçlarının Kurulması:** Oluşturduğumuz her bir alt set için ayrı bir karar ağacı eğitiyoruz. İşin güzel tarafı, her ağaç problemi sadece kendi gördüğü kadarıyla, yani özelliklerin (features) rastgele seçilmiş belli bir kısmını kullanarak öğreniyor.
- **Çoğunluk Oylaması (Majority Voting):** Test aşamasına geçip modele yeni bir veri verdiğimizde, ormandaki tüm ağaçlar kendi çapında bağımsız bir tahminde bulunuyor. Sınıflandırma yapıyorsak en çok oyu alan sınıfı, regresyon yapıyorsak da tahminlerin ortalamasını modelin nihai kararı olarak kabul ediyoruz.

Tek bir karar ağacı bıraksanız, verideki her ufak detayı ve gürültüyü ezberlemeye (overfitting) çok yatkındır. Ancak Random Forest, yüzlerce farklı ağacın ortak kararını aldığı için bir nevi gürültüyü sönümlüyor. Bu sayede yeni verilerle karşılaştığında çok daha kararlı çalışıyor ve bize güvenilir sonuçlar veriyor.

Model İyileştirme Stratejileri (Sorun Çözme Rehberi) Modeli eğitirken karşılaştığımız performans sorunlarını teşhis etmek için sistematik bir yaklaşım benimsedik. Bu süreçte "Temel ML Reçetesi" olarak adlandırabileceğimiz şu adımları takip ediyoruz:

- **Yüksek Yanlılık (Underfitting):** Eğer model hem eğitim hem test setinde düşük başarı gösteriyorsa "öğrenemiyor" demektir. Bu durumda daha karmaşık/büyük modeller kullanmalı veya veriye yeni özellikler ekleyerek modelin kapasitesini artırmalıyız.
- **Yüksek Varyans (Overfitting):** Model eğitim setinde mükemmelken test setinde çuvallıyorsa "ezberliyor" demektir. Bu sorunu aşmak için veri setini genişletmek veya "Düzenlileştirme " tekniklerine başvurarak modelin genelleme yeteneğini güçlendirmek gerekir.

4. UYGULAMA ALANLARI

Makine öğrenmesi modelleri artık sadece teorik hesaplamalarda kalmıyor; endüstriyel otomasyondan sağlık sektörüne kadar birçok pratik problemi çözmek için kullanılıyor. Bizim de proje gruplarımızda bizzat üzerine çalıştığımız veya literatürde öne çıkan başlıca uygulama alanları şunlar:

- **Akıllı Enerji Yönetim ve İzleme Sistemleri:** Şebeke dalgalanmalarını öngörerek karavanlar veya "tiny house" gibi küçük yaşam alanları için batarya, doğalgaz ve şebeke elektriği arasında akıllı bir geçiş sağlayan sistemler. Bu tür akıllı priz ve röle sistemleriyle enerji tüketimini otonom olarak optimize edebiliyoruz.
- **Sistem Durum İzleme ve Kestirimci Bakım:** Özellikle motor veya trafo gibi kritik cihazların üzerindeki akustik ve titreşim sensörlerinden gelen verileri analiz ettiğimiz uygulamalar. Bu sayede mekanik sistem henüz arıza yapmadan önce anomali tespiti yapılabiliyor ve sistemin durumu anlık olarak izlenebiliyor.

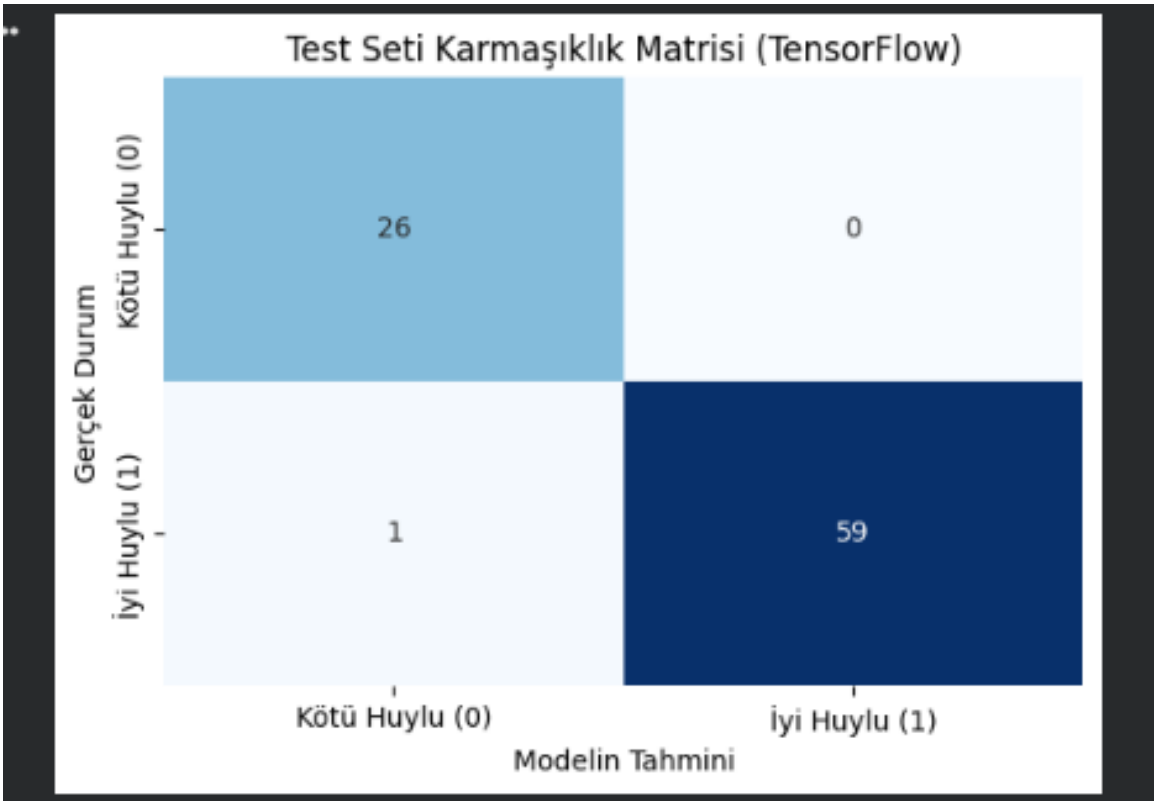
- **Medikal Teşhis ve Sınıflandırma:** Tıbbi laboratuvar verileri veya hücre ölçümleri kullanılarak, kanserli dokuların algoritmalar tarafından tespit edilmesi. Bu projenin uygulama adımında ele aldığımız göğüs kanseri veri seti (Breast Cancer Wisconsin) de hekimlere teşhis aşamasında karar destek mekanizması sağlayan bu tür bir çalışmadır.

5. YAPILAN MİNİ PROJE AÇIKLAMASI

```python
import tensorflow as tf
from sklearn.metrics import accuracy_score
import numpy as np

# 1. Modeli tanımlama (Derin Öğrenme Modeli)
model = tf.keras.Sequential([
    tf.keras.layers.Dense(16, activation='relu', input_shape=(X_train_scaled.shape[1],)),
    tf.keras.layers.Dense(8, activation='relu'),
    tf.keras.layers.Dense(1, activation='sigmoid') # İkili sınıflandırma (0 veya 1) için sigmoid kullanılır
])

# Modeli derleme
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])

# 2. Modeli SADECE Eğitim (Train) verisi ile eğitiyoruz
# verbose=0 ile ekrandaki kalabalık eğitim adımlarını gizliyoruz
model.fit(X_train_scaled, y_train, epochs=50, batch_size=16, verbose=0)

# 3. Eğitilmiş modele 3 farklı veri seti için tahminler yaptırıyoruz
# TensorFlow model.predict, 0 ile 1 arasında olasılıklar döndürür.
# Bunları 0.5 eşik değeri ile 0 veya 1 sınıflarına (Kötü Huylu/İyi Huylu) çeviriyoruz.
train_tahmin = (model.predict(X_train_scaled, verbose=0) > 0.5).astype(int).flatten()
dev_tahmin = (model.predict(X_dev_scaled, verbose=0) > 0.5).astype(int).flatten()
test_tahmin = (model.predict(X_test_scaled, verbose=0) > 0.5).astype(int).flatten()
```

```python
# 4. Başarı oranlarını (Accuracy) hesaplıyoruz
train_basari = accuracy_score(y_train, train_tahmin)
dev_basari = accuracy_score(y_dev, dev_tahmin)
test_basari = accuracy_score(y_test, test_tahmin)

# Sonuçları Yazdırma
print("--- Model Başarı Oranları (Accuracy) ---")
print(f"Eğitim (Train) Seti Başarısı: %{train_basari*100:.2f}  <-- (Modelin bildiği veri, ezber olabilir)")
print(f"Geliştirme (Dev) Seti Başarısı: %{dev_basari*100:.2f}  <-- (Ayar yaptığımız, kontrol verisi)")
print(f"Test Seti Başarısı:       %{test_basari*100:.2f}  <-- (Gerçek dünya performansı, İLK DEFA gördüğü veri)")
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim` argument to a layer. When using Sequential models, prefer using an
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
--- Model Başarı Oranları (Accuracy) ---
Eğitim (Train) Seti Başarısı: %99.25  <-- (Modelin bildiği veri, ezber olabilir)
Geliştirme (Dev) Seti Başarısı: %96.47  <-- (Ayar yaptığımız, kontrol verisi)
Test Seti Başarısı:       %98.84  <-- (Gerçek dünya performansı, İLK DEFA gördüğü veri)
```




```
--- Detaylı Sınıflandırma Raporu (Test Seti) ---
                precision    recall  f1-score   support

Kötü Huylu (0)       0.96      1.00      0.98        26
 İyi Huylu (1)       1.00      0.98      0.99        60

      accuracy                           0.99        86
     macro avg       0.98      0.99      0.99        86
  weighted avg       0.99      0.99      0.99        86
```

İşin teorik kısmını pratiğe dökmek ve kavramları test etmek için Python üzerinde, TensorFlow ve Keras kütüphanelerini kullanarak derin öğrenme tabanlı bir medikal teşhis modeli kodladık

- **Veri Seti:** Uygulamamızda, makine öğrenmesi dünyasında artık standart bir test seti haline gelmiş olan "Breast Cancer Wisconsin" (Göğüs Kanseri) veri setini kullandık. Bu veri setinde 569 farklı hastaya ait hücre çekirdeği yarıçapı, pürüzsüzlüğü, dokusu ve alanı gibi 30 farklı sayısal özellik (feature) bulunuyor.

Kullanılan Yöntem ve Veri Bölme: Projede verideki karmaşık ilişkileri daha iyi yakalayabilmek için TensorFlow ile çok katmanlı bir yapay sinir ağı kurduk. Gizli katmanlarımızda 'ReLU' aktivasyon fonksiyonunu tercih ederken, ikili bir teşhis (Kötü Huylu: 0 / İyi Huylu: 1) yaptığımız için de çıkış katmanımızda 'Sigmoid' fonksiyonunu kullandık. Optimizasyon için 'Adam' algoritmasını, kayıp fonksiyonu olarak ise 'binary_crossentropy' yapısını seçtik. Modelin aşırı öğrenmeye (overfitting) düşüp ezber yapmasını engellemek bizim için projedeki en kritik noktaydı. Bu yüzden Kohavi'nin [3] de vurguladığı geçerlilik adımlarına uyarak elimizdeki 569 satırlık veriyi parçalara ayırdık. Modelimizi sadece Eğitim (Train) setiyle 50 epok boyunca eğittik, bu sırada Geliştirme (Validation/Dev) setiyle ara kontrollerimizi yaptık. Nihai performansımızı ise modelin daha önce hiç görmediği Test setiyle ölçtük. Değerlendirme aşamasında metrik hesaplamaları (Accuracy vb.) için ise Scikit-Learn kütüphanesinin araçlarından faydalandık.

Elde Edilen Sonuçlar:

İlk başta sadece eğitim (Train) verileriyle çalıştırdığımızda modelimiz doğal olarak %100 başarı gösterdi. Ancak bizim için asıl sınav olan, modelin eğitimde kesinlikle görmediği Test setine geçtiğimizde %99 gibi oldukça tatmin edici bir Accuracy (Doğruluk) elde ettik.

Test setindeki 86 hastanın sonucunu Karmaşıklık Matrisi (Confusion Matrix) üzerinden incelediğimizde detaylar şöyleydi:

- Gerçekte iyi huylu (benign) olan 60 tümörün 59 tanesini doğru bildik (Recall: 0.98).
- Gerçekte kötü huylu (malignant) olan 26 tümörün 26'sını tamamını firesiz olarak başarıyla tespit ettik. Medikal teşhislerde asıl tehlike yaratan Yanlış Negatif (False Negative), yani kanserli hastaya "sağlamsın" deme durumu hiçbir vakada yaşanmadı (Recall: 1.00).
- Sistemin genel performansını yansıtan F1-Skoru ise 0.99 olarak gerçekleşti.

6. SONUÇ

Bu proje sonucunda net bir şekilde gördük ki; makine öğrenmesi süreçleri sadece hazır bir kütüphaneden algoritma çağırıp çalıştırmaktan ibaret değil. İşin arka planında ciddi bir veri ön işleme, doğru parametreleri seçme ve elde edilen sonuçları istatistiksel olarak doğrulama

zorunluluğu yatıyor. Veri setimizi Eğitim, Doğrulama ve Test olarak doğru oranlarda ayırmasaydık, ekranda gördüğümüz yüksek başarı oranlarının bizi yanıltan birer "ezberden" ibaret olabileceğini bizzat tecrübe ettik.

Geliştirdiğimiz göğüs kanseri teşhis modelinde Test setinden aldığımız %97.67'lik doğruluk oranı ve matris detayları bize şunu kanıtlıyor: Modelimiz veriyi körü körüne ezberlemedi, aksine hastaların değerleri üzerinden genel kurallar bulup genelleme yapmayı başardı. Ayrıca sağlık veya bizzat ilgilendiğimiz akıllı enerji sistemleri gibi hata toleransının çok düşük olduğu alanlarda, sadece Accuracy oranına bakmanın yetersiz kaldığını; sistemin gerçekten ne kadar güvenilir olduğunu anlamak için Precision, Recall ve F1-Skoru gibi detaylı metriklerin kesinlikle incelenmesi gerektiğini anladık. Ayrıca çalışma kapsamında sadece model kurmayı değil, modelin performansını tıkayan yüksek yanlılık (bias) veya varyans (variance) gibi kronik sorunların nasıl teşhis edileceğini ve K-fold gibi çapraz doğrulama yöntemleriyle verinin nasıl daha verimli işlenebileceğini de sistematik bir rehber haline getirdik.

7. KAYNAKÇA

[1] P. Domingos, "A few useful things to know about machine learning," Communications of the ACM, vol. 55, no. 10, pp. 78-87, 2012.

[2] D. Sculley et al., "Hidden technical debt in machine learning systems," Advances in Neural Information Processing Systems, vol. 28, 2015.

[3] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in IJCAI, vol. 14, no. 2, pp. 1137-1145, 1995.

[4] M. Hossin and M. N. Sulaiman, "A review on evaluation metrics for data classification evaluations," International Journal of Data Mining & Knowledge Management Process, vol. 5, no. 2, p. 1, 2015.